NOR Memory Access

Name: Mohan Ram SanthoshKumar
CDSID: MSANTHOS

Table of Contents

1 Introduction:

This C program transcends basic communication with a W25Q64 NOR flash memory on your Raspberry Pi Pico. It empowers you to manage data storage by selectively erasing sectors, writing data to specific pages within those sectors, and verifying the written data. This paves the way for building versatile applications on your Pico, enabling persistent storage of system configurations, user preferences, data logs, or even firmware updates, all while offering flexibility in data manipulation and the potential for future enhancements to create a robust data storage solution for your embedded projects.

2 Scope:

This program goes beyond basic NOR memory interaction, serving as a building block for various data storage applications. It enables erasing specific sectors, targeted data writing, and reliable read verification. You can extend it to store system configurations, log sensor data, hold bootloader code, or even implement firmware updates on your Pico, unlocking the potential of NOR flash for persistent data management in your embedded projects.

3 Prerequisite:

3.1 Hardware:

- **Raspberry Pi Pico:** A budget-friendly, high-efficiency microcontroller board featuring a dual-core Arm Cortex-M0+ processor.
- **Personal Computer (PC):** To use the serial console application and connect with the Raspberry Pi Pico.
- **USB Power Cable:** A micro-USB cable that powers the Pico and enables USB to Serial communication.
- **W25Q64 (NOR device)**

3.2 Software:

- **PuTTY:** A free, open-source terminal emulator supporting various network protocols, including serial communication. It will be used to monitor the UART output from the Pico.
- **Pico SDK:** A comprehensive software development kit for the Raspberry Pi Pico, offering libraries and tools for C/C++ programming.
- **GCC (GNU Compiler Collection):** A compiler system supporting various programming languages, essential for compiling the C program.
- **Device Driver:** Ensure the USB to Serial driver is installed on the PC to recognize the Pico as a serial device.

4 Connection Diagram:

Pico Pin	GPIO	Breakout Label	Signal
Pin 21	GP16	DO / MISO	MISO
Pin 22	GP17	CS / CE	Chip Select
Pin 24	GP18	CLK / SCK	Clock
Pin 25	GP19	DI / MOSI	MOSI
Pin 36	3.3V	VCC / 3V3	Power
Pin 38	GND	GND	Ground

5 Working Principle:

· SPI Communication:

§ The Pico communicates with the W25Q64 via the SPI interface. The Pico acts as the master, and the W25Q64 as the slave.

§ SPI involves four signals: SCK (clock), MOSI (Master Out Slave In), MISO (Master In Slave Out), and CS (Chip Select).

· Erase Sector:

§ The W25Q64 allows erasing sectors of its memory. This is necessary before writing new data to the memory.

· Write Data:

§ Data is written to the W25Q64 memory after a sector has been erased.

· Read Data:

§ Data is read back from the W25Q64 memory and printed to the UART console.

6 Application code:

Experiment6.c

```c
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/spi.h"

// Hardware Pin Definitions
#define SPI_PORT spi0
#define PIN_MISO 16
#define PIN_CS   17
#define PIN_SCK  18
#define PIN_MOSI 19

// W25Q64 Instruction Set
#define CMD_WRITE_ENABLE 0x06
#define CMD_SECTOR_ERASE 0x20
#define CMD_PAGE_PROGRAM 0x02
#define CMD_READ_DATA    0x03
#define CMD_READ_STATUS  0x05
#define CMD_JEDEC_ID     0x9F

// --- Helper Functions ---

static inline void cs_select() {
    asm volatile("nop \n nop \n nop");
    gpio_put(PIN_CS, 0);
    asm volatile("nop \n nop \n nop");
}

static inline void cs_deselect() {
    asm volatile("nop \n nop \n nop");
    gpio_put(PIN_CS, 1);
    asm volatile("nop \n nop \n nop");
}

// Check if the flash is busy with an internal write/erase
void wait_busy() {
    uint8_t status;
    uint8_t cmd = CMD_READ_STATUS;
    do {
        cs_select();
        spi_write_blocking(SPI_PORT, &cmd, 1);
        spi_read_blocking(SPI_PORT, 0, &status, 1);
        cs_deselect();
    } while (status & 0x01); // Bit 0 is BUSY
}

// 1. VALIDATION: Read Manufacturer ID
void flash_read_id() {
```

```
    uint8_t cmd = CMD_JEDEC_ID;
    uint8_t response[3];

    cs_select();
    spi_write_blocking(SPI_PORT, &cmd, 1);
    spi_read_blocking(SPI_PORT, 0, response, 3);
    cs_deselect();

    printf("\n--- Chip Validation ---\n");
    printf("Manufacturer ID: 0x%02X (Winbond is 0xEF)\n",
       response[0]);
    printf("Memory Type:     0x%02X\n", response[1]);
    printf("Capacity ID:     0x%02X (64Mbit is 0x17)\n", response
       [2]);

    if (response[0] == 0xEF) {
        printf("STATUS: Hardware Connection Verified!\n");
    } else {
        printf("STATUS: Connection FAILED. Check MISO/MOSI wiring
           .\n");
    }
}

void flash_erase_sector(uint32_t addr) {
    uint8_t enable_cmd = CMD_WRITE_ENABLE;
    cs_select();
    spi_write_blocking(SPI_PORT, &enable_cmd, 1);
    cs_deselect();

    uint8_t erase_buf[4] = {CMD_SECTOR_ERASE, (addr >> 16) & 0xff
       , (addr >> 8) & 0xff, addr & 0xff};
    cs_select();
    spi_write_blocking(SPI_PORT, erase_buf, 4);
    cs_deselect();
    wait_busy();
}

void flash_write_page(uint32_t addr, uint8_t *data, size_t len) {
    uint8_t enable_cmd = CMD_WRITE_ENABLE;
    cs_select();
    spi_write_blocking(SPI_PORT, &enable_cmd, 1);
    cs_deselect();

    uint8_t prog_buf[4] = {CMD_PAGE_PROGRAM, (addr >> 16) & 0xff,
        (addr >> 8) & 0xff, addr & 0xff};
    cs_select();
    spi_write_blocking(SPI_PORT, prog_buf, 4);
    spi_write_blocking(SPI_PORT, data, len);
    cs_deselect();
    wait_busy();
}
```

```
void flash_read(uint32_t addr, uint8_t *buf, size_t len) {
    uint8_t read_cmd[4] = {CMD_READ_DATA, (addr >> 16) & 0xff, (
       addr >> 8) & 0xff, addr & 0xff};
    cs_select();
    spi_write_blocking(SPI_PORT, read_cmd, 4);
    spi_read_blocking(SPI_PORT, 0, buf, len);
    cs_deselect();
}

// --- Main Program ---

int main() {
    stdio_init_all();

    // 1. Initialize SPI at 1MHz
    spi_init(SPI_PORT, 1000 * 1000);
    gpio_set_function(PIN_MISO, GPIO_FUNC_SPI);
    gpio_set_function(PIN_SCK,  GPIO_FUNC_SPI);
    gpio_set_function(PIN_MOSI, GPIO_FUNC_SPI);

    // 2. Manual Chip Select Setup
    gpio_init(PIN_CS);
    gpio_set_dir(PIN_CS, GPIO_OUT);
    gpio_put(PIN_CS, 1); // High = Deselected

    // Wait for USB Serial
    while (!stdio_usb_connected()) {
        sleep_ms(100);
    }

    printf("\n========================================");
    printf("\n   W25Q64 SPI FLASH VALIDATION TOOL");
    printf("\n========================================\n");

    // STEP A: Validate Connection via JEDEC ID
    flash_read_id();

    // STEP B: Perform Real Write/Read Cycle
    uint32_t addr = 0x000000;
    uint8_t test_data[] = "Pico_Verify_99"; // Change this to
       anything to test!
    uint8_t rx_buffer[20] = {0};

    printf("\n[Step 1] Erasing Sector...");
    flash_erase_sector(addr);
    printf(" Done.");

    printf("\n[Step 2] Writing Data: '%s'...", test_data);
    flash_write_page(addr, test_data, sizeof(test_data));
    printf(" Done.");
```

```
    printf("\n[Step 3] Reading Data from Flash...");
    flash_read(addr, rx_buffer, sizeof(rx_buffer));

    printf("\n\nFINAL RESULT: %s", rx_buffer);
    printf("\n========================================\n");

    while (1) {
        tight_loop_contents();
    }

    return 0;
}
```

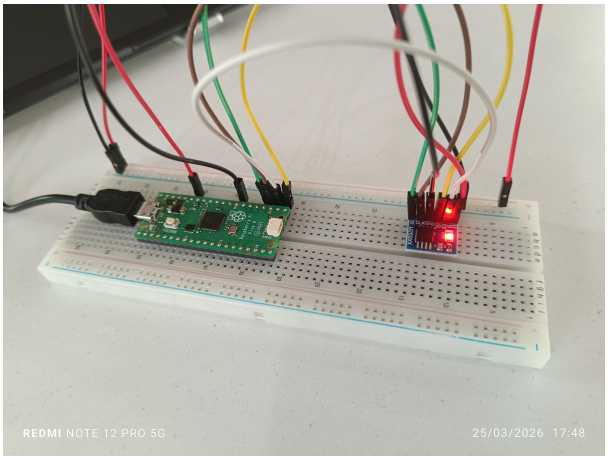
CMakeLists.txt

```
# Generated Cmake Pico project file

cmake_minimum_required(VERSION 3.13)

set(CMAKE_C_STANDARD 11)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

# Initialise pico_sdk from installed location
# (note this can come from environment, CMake cache etc)

# == DO NOT EDIT THE FOLLOWING LINES for the Raspberry Pi Pico VS
    Code Extension to work ==
if(WIN32)
    set(USERHOME $ENV{USERPROFILE})
else()
    set(USERHOME $ENV{HOME})
endif()
set(sdkVersion 2.2.0)
set(toolchainVersion 14_2_Rel1)
set(picotoolVersion 2.2.0-a4)
set(picoVscode ${USERHOME}/.pico-sdk/cmake/pico-vscode.cmake)
if (EXISTS ${picoVscode})
    include(${picoVscode})
endif()
#
   ================================================================================

set(PICO_BOARD pico CACHE STRING "Board type")

# Pull in Raspberry Pi Pico SDK (must be before project)
include(pico_sdk_import.cmake)

project(Experiment6 C CXX ASM)
```

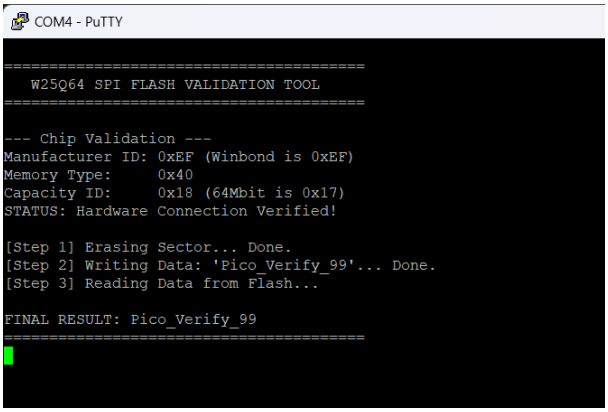
```
# Initialise the Raspberry Pi Pico SDK
pico_sdk_init()

# Add executable. Default name is the project name, version 0.1

add_executable(Experiment6 Experiment6.c )

pico_set_program_name(Experiment6 "Experiment6")
pico_set_program_version(Experiment6 "0.1")

# Modify the below lines to enable/disable output over UART/USB
pico_enable_stdio_uart(Experiment6 1)
pico_enable_stdio_usb(Experiment6 1)

# Add the standard library to the build
target_link_libraries(Experiment6
        pico_stdlib
        hardware_spi)

# Add the standard include files to the build
target_include_directories(Experiment6 PRIVATE
        ${CMAKE_CURRENT_LIST_DIR}
)

pico_add_extra_outputs(Experiment6)
```

7 Output Image:

Hardware:



Software:



8 Appendix:

- **Baud Rate:** This is the speed at which data is transferred in a communication channel. For this experiment, it's set to 115200 bits per second.
- **COM Port:** This is the interface through which the PC communicates with Pico. There is a need to select the correct COM port in PuTTY.
- **PuTTY Configuration:** Make sure the correct COM port and baud rate are selected in PuTTY for successful communication. Other settings like data bits, stop bits, and parity should also match Pico's configuration.